

SYSTEM DESIGN INTERVIEW

Design Slack

workspace + channel + DM messaging with persistent history, full-text search, and integrations

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01** · Problem Statement
- 02** · Requirements
- 03** · Capacity Estimation
- 04** · Domain Model
- 05** · High-Level Architecture
- 06** · Real-Time Delivery (WebSockets + RTM)
- 07** · Storage Schema
- 08** · Message Send & Fan-out
- 09** · Search
- 10** · Notifications & Unread Counters
- 11** · Threads

- 12** · Integrations & Bots
- 13** · Failure Modes & Recovery
- 14** · Slack vs WhatsApp
- 15** · Design Trade-offs
- 16** · Interview Playbook

01

Problem Statement

Requirements and scope

Design **Slack**: a workspace-based team-messaging service. Users belong to one or more **workspaces**; each workspace contains **channels** (public, private, DMs, group-DMs). Messages are delivered in real time over WebSockets, persisted forever, and indexed for full-text search. Bots and integrations consume an event bus.

Clarifying Questions

Question	Answer
Scale?	~20M DAU, ~1.5B messages/day globally
Topology?	Workspace → channels → messages; user in N workspaces
Channel types?	Public, private, 1:1 DM, group-DM (≤9 users), threads
Delivery?	Real-time fan-out to online clients; push for offline
History?	Persistent forever (workspace pays for retention tier)
Search?	Full-text per-workspace; near-real-time (~1–5s freshness)
Encryption?	TLS in transit, AES at rest; not E2EE — server reads to index/search
Integrations?	Apps, Events API webhooks, slash commands, RTM bots
Multi-region?	Yes; workspace pinned to home region for compliance

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Send message	POST to channel/DM; deliver to all subscribed online clients
Receive in real time	WebSocket push within 1s end-to-end (p99)
Persistent history	Forever; scrollbar, pagination by ts cursor
Threads	Sub-conversation rooted at parent_ts; UI merges into channel
Search	Per-workspace inverted index; user/channel/time/keyword filters
Unread + mentions	Per-channel unread counter; @mentions and DMs trigger push
Presence	Online / away / DND status visible per workspace
Integrations	Events API webhooks, slash commands, RTM bots, file uploads

Non-Functional Requirements

Requirement	Target
Availability	99.99% — chat outages are highly visible
Latency (send → deliver)	p50 < 200ms, p99 < 1s within region
Search freshness	1–5 sec from send to searchable
Durability	11 nines; never lose a persisted message
Throughput	~17K msgs/sec avg, ~50K peak; ~5M concurrent WebSockets
Storage	~1.5 TB/day raw + indexes; multi-year retention
Security	TLS, AES-256 at rest, SSO/SAML, audit log; not E2EE

03

Capacity Estimation

Math for scale

Traffic

- DAU: **20M** (active users sending or reading on a given day)
- Messages/day: **1.5B** (≈ 75 messages per DAU)
- Average write rate: $1.5B / 86,400 \approx \mathbf{17,360\ msgs/sec \approx 17K/sec}$
- Peak (Mon 9am US/EU overlap): $\sim 3\times$ average $\approx \mathbf{50K\ msgs/sec}$
- Reads (history scrollbar + live delivery fan-out): each message fans out to ~ 10 subscribers on average
→ **$\sim 170K\ deliveries/sec$ avg, $\sim 500K$ peak**
- Concurrent WebSocket connections: $\sim 25\%$ of DAU online simultaneously → **$\sim 5M$ live sockets**

Storage

- Average message size: **$\sim 1\ KB$** (text + JSON envelope; files are separate)
- Raw daily volume: $1.5B \times 1\ KB = \mathbf{1.5\ TB/day}$
- Search index overhead: $\sim 30\%$ of raw text → **$\sim 0.5\ TB/day$ additional**
- Replication factor 3 → **$\sim 6\ TB/day$ physical**
- Annual: **$\sim 2.2\ PB/year$ raw** ($\approx 6.5\ PB$ replicated). Years of history per workspace.

Cache & Memory

- Hot recent window: last 7 days of channels users actively read $\approx \mathbf{\sim 10\ TB}$ across the fleet
- Presence + unread counters in Redis: $20M\ users \times \sim 200\ bytes = \mathbf{\sim 4\ GB}$ hot data
- WebSocket session state: $\sim 5M\ sockets \times 1\ KB\ metadata = \mathbf{\sim 5\ GB}$ across RTM gateways

KEY

Key Numbers

20M DAU · 1.5B msgs/day · 17K avg / 50K peak msgs/sec · $\sim 5M$ live WebSockets · 1.5 TB raw/day · $\sim 2.2\ PB/year$. Persistent forever.

04

Domain Model

Workspace, channel, user, message

Slack's mental model is a tree: a **user** belongs to N **workspaces**; each workspace contains **channels**; each channel contains **messages**; some messages have a **thread** of replies. DMs and group-DMs are simply special channels with implicit membership.

Entities

Entity	Notes
Workspace	Tenant boundary; billing, retention, SSO scoped here
User ↔ Workspace	Many-to-many via membership; user has one identity per workspace
Channel	Belongs to exactly one workspace; type ∈ {public, private, dm, mpdm}
Membership	(workspace_id, channel_id, user_id) → joined_at, last_read_ts
Message	(workspace_id, channel_id, ts) — ts is server-assigned monotonic
Thread	Messages with parent_ts != NULL; thread_ts = root parent's ts
Reaction / file / pin	Side tables keyed on (channel_id, ts)

Why workspace_id Is the Shard Key

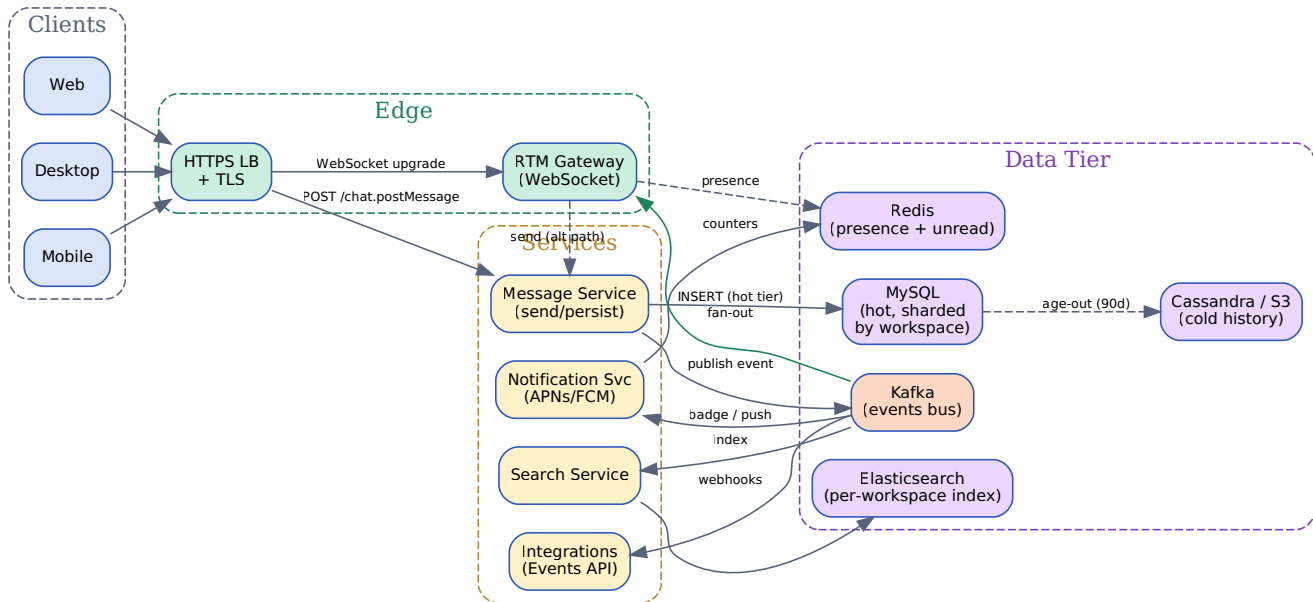
- **Tenant isolation:** all queries are scoped by workspace; cross-workspace queries are rare
- **Compliance:** a workspace can be pinned to a region (EU) for data residency
- **Billing & retention:** applied per workspace; co-locating its data simplifies lifecycle
- **Hot-shard risk:** a giant workspace (50K+ users) may need sub-sharding by channel_id

05

High-Level Architecture

System overview

Clients hold a long-lived WebSocket to an **RTM gateway**. Sends are POSTed over HTTPS (or sent on the WebSocket) to the **Message Service**, which persists, then publishes a **fan-out event** on Kafka. RTM gateways consume that event and push to subscribed sockets. Search, integrations, and notifications tap the same Kafka stream.



High-level architecture: WebSocket gateways front the clients; Kafka decouples persistence from fan-out, search, and integrations.

Component Responsibilities

- **RTM Gateway:** terminates WebSockets; tracks per-socket channel subscriptions; pushes events from Kafka to clients
- **Message Service:** validates send, assigns server ts, persists to MySQL, publishes to Kafka
- **Kafka:** single source of truth for the fan-out — keeps the hot send path linear and decouples consumers
- **Search Service:** consumes Kafka and indexes into per-workspace Elasticsearch shards
- **Notification Service:** updates unread/badge counters in Redis; fires APNs/FCM for mentions and DMs
- **Integrations:** Events API delivery, slash command dispatch, bot token management
- **Cold tier:** messages older than ~90 days move to Cassandra/S3; reads served on demand

06

Real-Time Delivery (WebSockets + RTM)

Pushing messages to online clients

Connection Lifecycle

1. Client logs in → calls `rtm.connect` → receives a WebSocket URL pinned to its workspace's home region
2. Client opens WebSocket; gateway authenticates token, loads **channel subscription set** for the user (channels they belong to)
3. Gateway registers (`socket_id` → `user_id`, `workspace_id`, `channel_set`) in its local in-memory map
4. Gateway also subscribes (in Redis or via Kafka consumer-group routing) to the relevant channel keys for fan-out
5. Heartbeat ping/pong every 10s; idle timeout 30s
6. On disconnect, client reconnects with `last_event_id` so the gateway replays missed events from a short buffer

Subscription Tracking

- Server-side per-socket channel set is the source of truth — clients never tell the server what to deliver
- On `conversations.join/leave`, gateway updates the in-memory map and persists to MySQL
- Cross-gateway events (a user in two devices) → both sockets receive the event independently
- Sticky load balancing keeps a workspace's sockets on a stable set of pods, improving Kafka consumer locality

Delivery to Online + Offline Users

- **Online:** Kafka event → RTM gateway looks up `channel_id` → all matching sockets get the JSON payload
- **Offline:** persistence already happened at MS; on reconnect the client requests `conversations.history?oldest=last_read_ts` and replays
- **Push:** if the message is a DM or contains an @mention, Notification Service issues an APNs/FCM push regardless of socket state
- **Backpressure:** if a client's send buffer is full for > 5s, gateway drops the socket; client reconnects with cursor and catches up via REST

INSIGHT

WebSocket vs Long-Poll

WebSockets win for Slack: full-duplex, one TCP connection per session, ~5M concurrent at our scale. Long-poll would multiply request volume by ~10× and add seconds of latency on every message. The cost is sticky-routing and connection-draining complexity during deploys.

07

Storage Schema

Hot, cold, and indexed tiers

Hot Messages Table (MySQL, sharded by workspace_id)

```
CREATE TABLE messages (  
  workspace_id BIGINT NOT NULL,  
  channel_id BIGINT NOT NULL,  
  ts DECIMAL(16,6) NOT NULL, -- '1714931523.000200'  
  user_id BIGINT NOT NULL,  
  text MEDIUMTEXT NOT NULL,  
  parent_ts DECIMAL(16,6) NULL, -- thread root  
  thread_ts DECIMAL(16,6) NULL, -- denorm root for query  
  edited_at TIMESTAMP NULL,  
  attachments JSON NULL,  
  reactions JSON NULL, -- summary only  
  PRIMARY KEY (workspace_id, channel_id, ts),  
  KEY idx_thread (workspace_id, thread_ts, ts),  
  KEY idx_user (workspace_id, user_id, ts)  
) ENGINE=InnoDB  
  PARTITION BY RANGE (ts) (...);  
  
-- Shard key: workspace_id → 256 logical shards via Vitess.  
-- Composite PK gives O(log n) channel scrollbar by ts cursor.
```

Membership / Last-Read

```
CREATE TABLE channel_members (  
  workspace_id BIGINT NOT NULL,  
  channel_id BIGINT NOT NULL,  
  user_id BIGINT NOT NULL,  
  joined_at TIMESTAMP NOT NULL,  
  last_read_ts DECIMAL(16,6) NOT NULL,  
  notify_pref ENUM('all','mentions','none') DEFAULT 'all',  
  PRIMARY KEY (workspace_id, channel_id, user_id),  
  KEY idx_user (workspace_id, user_id)  
);
```

Tiering Policy

Tier	Age	Store	Latency	Cost
Hot	0–90 days	MySQL (sharded, replicated)	~5 ms point read	\$\$\$
Warm	90d–2y	Cassandra (workspace_id, channel_id, ts)	~30 ms range scan	\$\$
Cold	2y+	S3 / Glacier columnar (Parquet by month)	~1–5 s on demand	\$
Search	all ages	Elasticsearch per-workspace index	~50 ms keyword	\$\$

Why a Composite PK on (workspace_id, channel_id, ts)

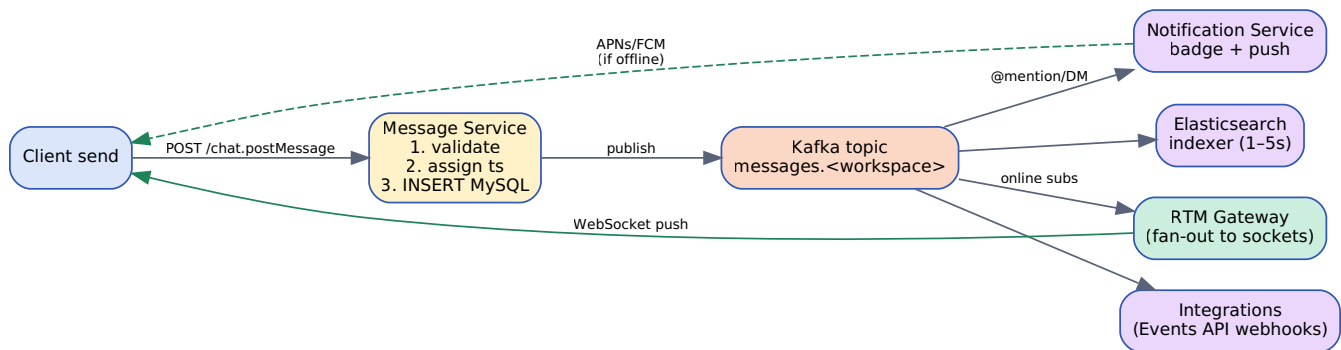
- Primary access pattern is “give me the next page of messages in channel C” — a range scan on (workspace, channel, ts)
- ts is a monotonic decimal — natural cursor for `WHERE ts < ? ORDER BY ts DESC LIMIT 50`
- All access is workspace-scoped, so the shard prefix matches the leading column of the PK
- Threads use a secondary index on (workspace_id, thread_ts, ts)

08

Message Send & Fan-out

From keystroke to subscribers

Lifecycle



Message lifecycle: persist before fan-out, then Kafka drives RTM, search, notifications, and integrations.

Step-by-Step

1. Client sends `POST /chat.postMessage { channel, text, client_msg_id }`
2. MS authenticates, checks channel membership, validates length, runs anti-abuse
3. MS assigns server timestamp `ts` (monotonic per channel) — this is the message's permanent ID
4. MS writes to MySQL (sharded by `workspace_id`). Synchronous — caller blocks until `fsync` replicates
5. MS publishes to Kafka topic `messages.`, partitioned by `channel_id`
6. RTM gateways consume → push to all subscribed online sockets within ~100 ms
7. Search indexer consumes → bulk-indexes into Elasticsearch (refresh interval 1s)
8. Notification Service consumes → bumps unread counters in Redis; fires APNs/FCM if @mention or DM
9. Integration Service consumes → posts Events API webhooks to subscribed apps
10. MS returns `{ ok, ts, channel }` to client; client replaces optimistic stub with confirmed message

Channel Fan-out (pseudocode)

```
# Runs in every RTM gateway pod, consuming the Kafka partition
# for the workspaces whose sockets this pod hosts.

def on_kafka_event(evt):
    # evt = { workspace_id, channel_id, ts, user_id, text, ... }
    sockets = local_index.subscribers_for(evt.workspace_id,
                                          evt.channel_id)

    if not sockets:
        return # nobody from this channel is online here
```

```

payload = serialize(evt)          # JSON, ~1 KB
for s in sockets:
    if s.user_id == evt.user_id and s.client_msg_id == evt.client_msg_id:
        continue                # don't echo to sender's same tab
    s.send_async(payload)         # non-blocking; drops on backpressure
    metrics.fanout_count.inc(len(sockets))

# local_index is built from ws-join events (channel_members) on connect.
# Lookup is O(1): { (ws_id, ch_id) -> set[socket] }.
# Average channel size at Slack is ~10 members, so most fan-outs touch ~3 sockets.

```

Idempotency & Ordering

- **client_msg_id** is a UUID generated by the client; MS dedupes on retry within 5 minutes
- **Per-channel ordering:** Kafka partition = channel_id → strict order on the consumer
- **ts ties:** if two clients send within the same microsecond, MS appends a serial suffix (e.g. 1714931523.000200)
- **Sender echo:** client gets the authoritative ts in the HTTP response and reconciles with what arrives over the WebSocket

09

Search

Per-workspace inverted index

Index Topology

- **One Elasticsearch index per workspace** (or per group of small workspaces, multi-tenant indexed by `workspace_id`)
- Big workspaces (500K+ messages/day) get dedicated indexes; small workspaces share a multi-tenant index
- **Refresh interval:** 1s — gives near-real-time search; trade some indexing throughput for freshness
- **Replication:** 1 primary + 1 replica per shard; rolling restarts safe
- **Time-based rollover:** roll into `messages-YYYY.MM` indexes; queries fan out to recent N indexes

Document Mapping

```
PUT messages-2026.05/_mapping
{
  "properties": {
    "workspace_id": { "type": "keyword" },
    "channel_id": { "type": "keyword" },
    "channel_type": { "type": "keyword" },
    "user_id": { "type": "keyword" },
    "ts": { "type": "date", "format": "epoch_second" },
    "thread_ts": { "type": "date", "format": "epoch_second" },
    "text": { "type": "text", "analyzer": "slack_std" },
    "mentions": { "type": "keyword" },
    "has_file": { "type": "boolean" },
    "is_private": { "type": "boolean" }
  }
}

# Custom analyzer: lowercases, splits on punctuation,
# preserves @mentions and #channels as single tokens,
# applies a unicode-aware stemmer.
```

Query Path

1. User types in search box → frontend hits `search.messages?q=...&channel=...&from=...`
2. Search Service rewrites the query: filters by `workspace_id`, by channels the user can read (private/DM ACL), by date range
3. Hits the workspace's index (or the time-rolled subset of monthly indexes)

4. Highlights matched terms; returns top 50 with (`channel`, `ts`) back-references
5. UI clicks on a hit → fetches the surrounding messages from MySQL/Cassandra by (`channel_id`, `ts`)

Search Freshness vs Cost

Refresh	Freshness	Indexing CPU	Notes
100ms	near-instant	very high	Slack does not need this; people don't search ms-old text
1s	1–5s end-to-end	moderate	Default; matches Slack's actual UX
30s	30–60s	low	Used for cold archive indexes; saves ~5× CPU
Manual	minutes+	very low	Used during bulk reindex (workspace migration)

ACL Enforcement

- Search Service *never* trusts the client's claimed channel list
- On query, it fetches the user's `channel_set` from membership cache and pushes it into the ES query as a filter
- Private channels and DMs the user isn't in are filtered out at query time, not deleted from the index
- This means leaving a private channel does not require re-indexing — only the membership cache changes

10

Notifications & Unread Counters

Badges, mentions, and pushes

Counter Model

- Per-channel unread for each member: `unread:{ws}:{ch}:{user}` in Redis
- Per-workspace badge: aggregate of unread channels with notifications enabled
- Per-mention counter: `mentions:{ws}:{user}` — drives the red badge bubble
- **Source of truth:** `last_read_ts` in MySQL (`channel_members`); Redis is a derived hot cache

Counter Update on New Message

```
# Notification Service consumer (Kafka → Redis)
def on_new_message(evt):
    members = members_cache.get(evt.workspace_id, evt.channel_id)
    is_dm = evt.channel_type in ("dm", "mpdm")
    mentioned = parse_mentions(evt.text)          # @user, @channel, @here

    pipe = redis.pipeline()
    for u in members:
        if u == evt.user_id:                      # sender's own msg
            continue
        # only bump unread if user hasn't read past this ts
        pipe.incr(f"unread:{evt.workspace_id}:{evt.channel_id}:{u}")
        if u in mentioned or evt.channel_type == "channel_at_channel":
            pipe.incr(f"mentions:{evt.workspace_id}:{u}")
    pipe.execute()

    # Push only for DMs and explicit mentions to avoid storm
    targets = mentioned if not is_dm else members - {evt.user_id}
    if len(targets) > AT_CHANNEL_PUSH_LIMIT:      # e.g. 500
        targets = throttle(targets)              # rate-limit @channel
    for u in targets:
        if presence_is_offline(u) or user_pref_always_push(u):
            push_apns_fcm(u, evt)
```

Push Storm Defense

- **@channel in 5K-member channels** would otherwise blast 5K APNs requests in one go
- Mitigation: per-user push rate limit (max ~10 push/min), batched into a single "new activity" notification on cooldown
- Workspaces can disable @channel for large rooms or require admin-only @channel

- Notification Service is horizontally scaled; Kafka partition count $\geq$ pod count ensures no single partition becomes the bottleneck during spikes

Mark-as-Read Path

1. Client sends `conversations.mark { channel, ts }`
2. Notification Service updates `last_read_ts` in MySQL and resets the Redis counter
3. Publishes a `read_state_changed` event so other devices of the same user update their badges
4. Across-device consistency: the user's other tabs see the badge clear within ~200 ms via their own WebSocket

11

Threads

Sub-conversations rooted at a parent

Data Model

- A reply has `parent_ts != NULL`; `thread_ts` equals the root parent's `ts`
- Threads are read via the secondary index (`workspace_id`, `thread_ts`, `ts`)
- Reply count + last-3-replies are denormalized onto the parent message for fast feed render
- Threads are first-class citizens in fan-out — a reply still goes to the channel's subscribers (with a `thread_ts` hint), but the UI only auto-opens it for thread participants

UI Merge Semantics

Mode	Behaviour	Notes
Default channel feed	Show parent only; "5 replies" stub	Avoid noise from busy threads
"Also send to channel"	Reply rendered inline in feed too	Sender opt-in per reply
Threads pane	Right-side pane lists all threads the user follows	Threads pinned by participation or @mention
All-unreads	Threads contribute to badge if user is a participant	Otherwise channel-level only

Thread Fan-out

- Same Kafka topic, same fan-out — all channel subscribers see the reply event
- RTM payload includes `thread_ts`, so the client routes it to the threads pane vs the main feed
- Notification Service additionally bumps `thread_unread: {ws}: {ch}: {thread_ts}: {user}` for participants

12

Integrations & Bots

Events API, slash commands, RTM bots

Why a Separate Event Bus

- Integrations are **untrusted external systems** (third-party apps, customer infra) — they can be slow, flaky, or misbehave
- If they were called inline from the send path, a slow webhook would stall messaging — unacceptable
- Solution: every persisted message is published to Kafka; the Integration Service consumes async with retries and per-app rate limits
- App outage doesn't affect the human chat experience — only delays bot replies

Integration Surface

Mechanism	Use	Latency
Events API webhook	Server-to-server: "a message was posted in channel X"	1–5s, retried with backoff
Slash command	Synchronous user-typed command → app HTTP endpoint	< 3s SLA, app must ack within 3s and use response_url for late replies
Interactive components	Button click / modal submit → app HTTP endpoint	< 3s SLA
RTM bots	Bot opens its own WebSocket; receives events	< 1s, behaves like a user
Incoming webhooks	External system POSTs into a channel	Treated as a regular send

Event Subscription

- Apps subscribe to event types (message.channels, app_mention, reaction_added, ...)
- Per-app filter applied at the Integration Service so Kafka can carry the firehose; the service does the projection
- **Delivery guarantees:** at-least-once with up to 3 retries over 1 hour; 4xx → drop with alert; 5xx → exponential backoff
- **Per-app rate limit:** 1 msg/sec/channel/app; bursts to 10 — protects laggy webhooks from queue buildup

TIP

Token Scopes

Slack apps use granular OAuth scopes (channels:read, chat:write, files:read). Events API verifies the request signing secret + timestamp to prevent replay. User tokens act as the user; bot tokens act as the bot identity inside channels it's invited to.

13

Failure Modes & Recovery

What can go wrong

Failure	Impact	Detection	Mitigation
RTM gateway pod down	~10–50K sockets dropped	Pod liveness; conn count drop	Clients reconnect with <code>last_event_id</code> ; LB drains pod; replay buffer (last 30s) covers the gap
MySQL primary down	Writes for that workspace shard stall	Replication lag spike; IO error	Orchestrator promotes replica ($\leq 30s$); Message Service returns 503; clients retry by <code>client_msg_id</code>
Kafka broker down	Send path slows; fan-out delayed	Producer ack timeouts	Multi-broker replication factor 3; producer retries to alt broker; consumers continue from offset
Elasticsearch shard unhealthy	Search degraded for that workspace	Cluster yellow/red	Replica promoted; queries fall back to neighbouring time-rolled indexes; hot reads continue (search is auxiliary)
Redis down (presence/counters)	Stale unread badges; presence dots flicker	Health check; PING fail	Read-through to MySQL <code>last_read_ts</code> ; counters rebuild lazily on next message; presence defaults to "active"
Push provider (APNs) outage	No mobile push for some users	APNs error rate alert	Queue with TTL 10 min; drop if not delivered (notification is best-effort); WebSocket delivery still works on next foreground
Workspace hot-shard	One huge tenant saturates a shard	QPS / conn skew	Sub-shard by <code>channel_id</code> ; pin to a dedicated shard pair; rate-limit @channel in giant rooms
Network partition (region)	Workspace pinned to region unreachable	Cross-region health check	Read-only replica in DR region; failover requires manual promotion to preserve ordering

Reconnection Protocol

- Client persists `last_event_id` per WebSocket session
- On reconnect: gateway checks if `last_event_id` is in the replay buffer (last ~30s of events)
- **Hit:** replay missed events directly over the new socket
- **Miss:** client falls back to `conversations.history` over REST, paginated by `ts` cursor

- Either way the client converges without dropping messages

WARNING

Graceful Degradation Order

If something has to give, in order: drop search freshness → drop badge accuracy → drop integrations → drop presence → never drop persisted message delivery. Sending and receiving real chat is the product; everything else is auxiliary.

14

Slack vs WhatsApp

Why these systems look different

Both deliver chat over the internet, but their constraints diverge on encryption, retention, and identity. Those choices ripple through the entire architecture.

Dimension	WhatsApp	Slack
Encryption	End-to-end (Signal protocol); server can't read	TLS in transit, AES at rest; server reads to index/search/audit
Identity	Phone number per user	Email + workspace membership; SSO/SAML
Persistence	Mostly device-local; server is a transient relay	Server-side history forever (workspace pays for retention)
Search	Local on device only	Server-side full-text (Elasticsearch) per workspace
Group size	<= 1024 members typical	Channels can have tens of thousands
Tenant model	No tenants — every user equal	Workspace (tenant) is the primary boundary
Compliance	Limited (E2EE prevents most)	Audit log, retention policy, eDiscovery, DLP
Push reliance	Mobile-first; push critical	Multi-device; WebSocket + push
Schema	Encrypted blobs; minimal server schema	Rich relational schema (workspace, channel, member, message, reaction, file...)
Storage tier	Mostly ephemeral; small server-side queue	Hot MySQL → warm Cassandra → cold S3; PB scale
Integrations	Limited (Business API)	Massive: Apps, Events API, slash commands, RTM bots

INSIGHT

The single decision that explains everything

WhatsApp chose E2EE → server is a relay → no server search → no eDiscovery → phone-number identity is enough. Slack chose server-side history → full-text search → admin/legal compliance → workspace tenancy → SSO. Once you pick encryption posture, the rest of the design is largely determined.

15

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Real-time transport	WebSocket	Single duplex connection; ~5M concurrent at our scale. Long-poll multiplies request count by ~10× and adds seconds of latency. Cost: sticky routing during deploys.
Fan-out	Async via Kafka after persistence	Adds ~50 ms vs in-line fan-out, but isolates slow integrations and search from the send hot path. In-line fan-out couples failures.
Shard key	workspace_id	Tenant isolation, residency, retention all align. Risk: huge workspaces become hot shards. Mitigation: sub-shard by channel_id when needed.
Hot store	MySQL with composite PK	Strong consistency, range scans by ts cursor are $O(\log n)$. Cost: sharding ops via Vitess. NoSQL would scale easier but lose transactions on read state.
Cold tier	Tiered to Cassandra/S3 after 90 days	Cuts hot-store cost ~10×; cold reads are slower but rare (most reads are recent). Migration job runs nightly per workspace.
Search refresh	1 second	Matches user expectation; ~2× the indexing CPU vs 30s. Sub-second would 5× the cost for no UX gain.
Search index per workspace	Yes (or grouped for small ones)	Tenant isolation, easy archival/migration. Cost: cluster has many indexes; mitigated by pooling small tenants.
Persistence model	Server-side (not E2EE)	Enables search, eDiscovery, integrations, audit. Cost: blast radius of a server breach is larger; offset with strict access control + per-workspace KMS keys.
Notification delivery	Best-effort APNs/FCM	Acceptable: WebSocket is the primary path; push is a nudge, not a guarantee. Avoids retry storms.
@channel in big rooms	Throttled / opt-in admin gate	Prevents push storms on 5K-member channels. Costs a tiny UX surprise; explained in onboarding.

16

Interview Playbook

How to present this

A clean Slack pitch frames the problem as “*persistent multi-tenant chat with server-side search*”, contrasts with WhatsApp early to anchor the encryption choice, then walks the send path end-to-end.

45-Minute Outline

1. **Intro (2 min)**: clarify scope — workspaces, channels, DMs, threads, server-side history, full-text search, integrations
2. **Anchor vs WhatsApp (2 min)**: server-side history → drives entire design
3. **Capacity (5 min)**: 20M DAU, 1.5B msgs/day, 17K avg / 50K peak, ~5M live WebSockets, 1.5 TB/day raw
4. **Domain model (3 min)**: workspace → channel → message; PK (workspace_id, channel_id, ts)
5. **High-level arch (4 min)**: RTM gateway, Message Service, Kafka, MySQL/Cassandra/S3, Elasticsearch, Redis
6. **Send + fan-out (8 min)**: persist before publish; Kafka per-channel ordering; pseudocode for fan-out
7. **Real-time delivery (5 min)**: WebSocket lifecycle, reconnection with last_event_id
8. **Search (5 min)**: per-workspace ES index, 1s refresh, ACL pushed into query
9. **Notifications + threads (5 min)**: Redis counters, push storm defense, thread sub-conversations
10. **Failures + trade-offs (5 min)**: hot shard, MySQL failover, graceful degradation order
11. **Wrap (1 min)**: recite memorized numbers, name the encryption-vs-search tension as the central choice

Key Talking Points

- “1.5B messages/day → 17K avg / 50K peak — peak is what we provision for”
- “workspace_id is the natural shard key — tenancy, residency, retention all align”
- “Persist *then* publish to Kafka — fan-out, search, integrations all become async readers”
- “WebSocket subscriptions are server-tracked, never client-claimed — that's how ACLs stay correct”
- “Search is auxiliary; chat must keep working if Elasticsearch is red”
- “Slack ≠ WhatsApp because of E2EE — that one decision determines tenancy, search, and compliance”

Common Follow-ups & Answers

- **Q: How do you guarantee per-channel ordering?** A: Single Kafka partition per channel_id; consumer-group keeps strict order; ts is server-assigned.

- **Q: A workspace has 100K users in #general — won't @channel kill us?** A: Per-user push rate limit, batched notifications, admin-only @channel option in big rooms.
- **Q: How are private channels kept private at search?** A: ACL filter pushed into the ES query; index isn't filtered; leaving a channel only updates the membership cache.
- **Q: An RTM pod dies with 50K sockets — what happens?** A: Clients reconnect with last_event_id; replay from the gateway's 30s buffer; fall back to history REST if older.
- **Q: How do you avoid duplicate sends on retry?** A: Client supplies a UUID client_msg_id; Message Service dedupes within 5 minutes; idempotent INSERT on (workspace, channel, client_msg_id).
- **Q: Why not E2EE like WhatsApp?** A: Slack's value prop includes server-side search and eDiscovery; E2EE would forfeit both. Compensate with strict KMS scoping per workspace.
- **Q: How do threads interact with channel ordering?** A: Replies fan out on the channel topic with thread_ts hint; the UI routes to the thread pane while the underlying log stays linear.

KEY

Memorize These Numbers

20M DAU · 1.5B msgs/day · 17K avg / 50K peak msgs/sec · ~5M live WebSockets · ~1 KB avg msg · 1.5 TB raw/day · ~2.2 PB/year · 1s search refresh · 90-day hot tier.